

ASSET TOKENIZATION: Decentralized Platform for Real-World

Asset Tokenization Using Blockchain

Omkar Bharath Pukale
Student

Department of CSE
AMC Engineering College,
Bengaluru, Karnataka, India
omkarbharathpukale@gmail.com

Premkumar M K
Assistant Professor

Department of CSE
AMC Engineering College,
Bengaluru, Karnataka, India
premkumarmk@gmail.com

Dr. V Mareeswari
Professor & HOD

Department of CSE
AMC Engineering College,
Bengaluru, Karnataka, India
mareesh.prasanna@gmail.com

Sahil Kumar Mishra
Student

Department of CSE
AMC Engineering College,
Bengaluru, Karnataka, India
sahilmishra7069@gmail.com

Sandesh G C
Student

Department of CSE
AMC Engineering College,
Bengaluru, Karnataka, India
sandysandesh148@gmail.com

Sanjitha A
Student

Department of CSE
AMC Engineering College,
Bengaluru, Karnataka, India
sanjitha0987@gmail.com

Abstract— Asset tokenization enables real-world assets to be represented as fractional digital tokens on a blockchain, allowing ownership to be divided into smaller units. This paper presents a web-based asset tokenization platform that integrates React.js, Node.js, Express.js, MongoDB, Solidity smart contracts, and MetaMask. The system uses ERC-20 tokens to represent fractional asset ownership and supports asset listing, token creation, token purchase, transfer, and ownership verification. Smart contracts are developed and tested using Hardhat and deployed on the Ethereum Sepolia test network for evaluation. The prototype demonstrates the integration of conventional web technologies with blockchain-based ownership management and provides a transparent mechanism for recording token transactions. The system is evaluated through functional, smart-contract, and integration testing to validate the proposed workflow.

Keywords— Asset Tokenization, Fractional Ownership, Blockchain Technology, Smart Contracts, Ethereum, Sepolia, ERC-20.

I. INTRODUCTION

Asset tokenization provides a way to represent ownership of physical assets through digital units recorded on a blockchain. It can divide assets such as property, commodities, and artwork into smaller portions, allowing people to invest without purchasing an entire asset. This approach can make participation easier for smaller investors while also supporting better movement of ownership in markets that are normally difficult to access. In the proposed system, smart contracts manage the creation,

transfer, and verification of asset tokens, reducing manual intervention and keeping transaction records consistent. The application uses React.js, Node.js, and MongoDB to support the user interface, application functions, and asset information. It also connects with an Ethereum test network for blockchain transactions. Together, these components provide a practical platform for listing assets, purchasing tokens, checking transactions, and demonstrating how blockchain can support simpler and more transparent asset ownership. Users can also view asset details and ownership information through the same platform. This makes asset investment and ownership easier to manage.

II. LITERATURE SURVEY

Recent studies examine asset tokenization from legal, financial, regulatory, and technical perspectives. Nishith Desai Associates [1] discusses legal and compliance issues in real-estate tokenization, including fractional ownership and regulatory requirements. Agur et al. [2] study how tokenization can reduce market inefficiencies while improving liquidity, settlement, and financial access. The World Economic Forum [3] highlights fractional ownership, programmability, transparency, and interoperability as important benefits of tokenized assets, while also identifying adoption and compliance challenges. Soni et al. [4] examine the role of distributed ledger technology in representing and transferring real-world assets, with emphasis on efficiency and accessibility. IFSCA [5] discusses the regulatory framework for real-world asset tokenization, covering issuance, trading, custody, and settlement requirements. Benedetti and Rodríguez-Garnica [6] examine tokenized assets and securities

and report potential benefits such as lower trading friction, transparency, wider market participation. Although the existing studies explain the legal, financial, and regulatory sides of tokenization in detail, they give less attention to building a working prototype that demonstrates the process from token creation to ownership transfer. This project focuses on that practical side. It implements fractional ownership, token transactions, and ownership management through Solidity smart contracts and a supporting web application. The aim is to show, through an actual prototype, how blockchain can be used to record ownership and manage transactions in a more transparent way.

III. SYSTEM ARCHITECTURE

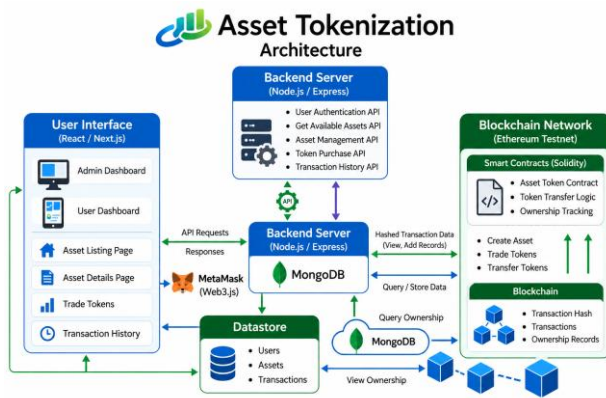


Figure 1. System Architecture and Component Integration

The proposed platform follows a layered architecture that separates user interaction, application processing, data persistence, blockchain operations, and wallet authorization. React.js forms the presentation layer through which users browse assets, view token holdings, and initiate transactions. The Node.js and Express.js backend provides application-level APIs and coordinates communication between the frontend, database, and blockchain components.

MongoDB stores application information such as user profiles, asset descriptions, images, and other off-chain data. Blockchain-related operations are handled through Solidity smart contracts deployed in the selected Ethereum test environment. MetaMask provides the wallet interface through which users approve and sign blockchain transactions.

This separation allows the application to use conventional database storage for frequently accessed information while maintaining token balances and blockchain transactions through smart contracts. The architecture also provides clear boundaries between application data and blockchain state, making individual components easier to test and modify.

IV. METHODS & IMPLEMENTATION

The implementation was carried out in four stages: system design, tokenization and marketplace development, application integration, and functional validation. It outlines the major phases involved in the project, starting from requirement analysis and system design to implementation, testing, and validation. Following a structured methodology helped ensure that the system was developed in an organized manner while meeting the intended objectives.

Requirement Analysis

The first step of the project was to identify the limitations of the existing asset ownership system. Traditional investment methods often require high capital, provide limited liquidity, and do not easily support fractional ownership. Based on these challenges, the system requirements were defined to develop a platform that enables secure asset tokenization, transparent ownership management, and easy trading of digital asset tokens.

Asset Fractionalization

The first step in the tokenization process is to divide the underlying real-world asset into smaller digital units. For this purpose, the project uses the ERC-20 token standard through the Property Token.sol smart contract. Instead of representing the complete property as a single ownership unit, the property is represented through multiple fungible tokens that can be distributed among different users. This allows an investor to hold only a portion of the asset rather than having to purchase the complete property. The fractionalization process therefore reduces the size of the individual ownership units and forms the basic foundation for the trading mechanism used in the platform.

Token Standard Selection

ERC-20 was selected for the prototype because the tokenized asset is represented as a set of interchangeable fractional units. The standard provides common functions for balances, transfers and approvals, which are sufficient for demonstrating fractional token ownership and exchange within the proposed platform.

ERC-1155 supports multiple token types within a common contract and can represent both fungible and non-fungible assets. This can be useful for systems managing several asset classes or token categories through a shared contract structure.

ERC-3643 is intended for permissioned token environments and includes mechanisms for identity

verification and transfer compliance. Such features are relevant to regulated asset tokenization, but they are outside the current prototype, which does not implement a complete on-chain KYC/AML and permissioned-transfer framework.

Therefore, ERC-20 was chosen for the present implementation because the primary objective was to demonstrate fungible fractional tokens, token transfers, and blockchain-based ownership tracking. ERC-1155 and ERC-3643 remain relevant alternatives for future versions requiring multi-token management or stronger identity and compliance controls.

Liquidity Mechanism Using an Automated Market Maker

The platform incorporates an Automated Market Maker (AMM) in which a liquidity pool contains asset tokens and ETH. Users can exchange tokens against the pool without requiring a matching counterparty. The exchange rate is determined from the pool reserves according to the implemented constant-product model.

$$X \times Y = K$$

where X denotes the asset-token reserve, Y denotes the ETH reserve, and K is the pool invariant. For an ETH input of ΔY and transaction fee rate f , the effective input is

$$\Delta Y_{\text{eff}} = \Delta Y(1 - f)$$

and the corresponding token output is

$$\Delta X = \frac{X \Delta Y_{\text{eff}}}{Y + \Delta Y_{\text{eff}}}.$$

The transaction fee is

$$F = \Delta Y f$$

and the execution price is

$$P_{\text{exec}} = \frac{\Delta Y}{\Delta X}.$$

These expressions describe how the pool reserves and configured transaction fee affect the token amount received in a swap.

The AMM contract also applies the configured transaction fee during swaps. The transaction is submitted through the user's wallet and the resulting token and ETH balances are updated by the smart contract. This approach provides an automated method for exchanging fractional asset tokens while keeping the final transaction state

verifiable on the blockchain.

The AMM is intended to demonstrate how tokenized assets can be exchanged through a programmable liquidity mechanism. The prototype does not claim that the AMM establishes market value for the underlying physical asset; the token price within the application is determined by the implemented trading mechanism and available liquidity.

Order Book Marketplace

In addition to the liquidity-pool mechanism, the platform incorporates an order-book based marketplace for users who want to specify the price at which they are willing to trade asset tokens. The order-book component is implemented through a dedicated smart contract that records the token quantity and requested price associated with a sell order.

A seller can deposit or offer a specified quantity of property tokens and define the corresponding asking price. A buyer can then select an available order and submit the required amount of ETH through the connected wallet. When the conditions defined by the marketplace contract are satisfied, the token and payment balances are updated accordingly.

For an order-book transaction, a sell order is represented by the tuple

$$O_s = (q_s, p_s, a_s)$$

where q_s is the number of tokens offered, p_s is the asking price per token, and a_s is the seller wallet address. A buyer submits a desired quantity q_b and a maximum acceptable price p_b . A trade is permitted when

$$p_b \geq p_s$$

The executed quantity is

$$q_{\text{exec}} = \min(q_b, q_s)$$

and the corresponding payment is

$$C = q_{\text{exec}} p_s.$$

After execution, the remaining quantities are updated as

$$q'_s = q_s - q_{\text{exec}}$$

and

$$q'_b = q_b - q_{exec}.$$

The smart contract completes the transfer only when the required token and payment conditions are satisfied.

The order-book mechanism provides a different trading method from the AMM. The AMM determines the exchange rate from liquidity-pool conditions, whereas the order book allows participants to specify prices for individual orders. Including both mechanisms gives the prototype two different approaches for demonstrating token exchange: automated liquidity-based trading and user-defined price-based trading.

The order-book implementation is evaluated as part of the prototype's transaction workflow. Its purpose is to demonstrate how blockchain smart contracts can maintain trade conditions and execute asset-to-token transfers without requiring the application database to act as the authoritative ownership ledger.

Smart Contract Development

The blockchain component of the platform is implemented using Solidity smart contracts that define the rules for token creation, token transfer, trading, and ownership management. The contracts form the blockchain-facing part of the application and are accessed through the wallet and application interface.

The property-token contract represents fractional units of an asset using the ERC-20 token standard. Each token represents a defined fractional unit within the prototype's asset model. Token balances are associated with blockchain wallet addresses, allowing ownership changes to be recorded through blockchain transactions.

The trading functionality is handled by the implemented marketplace contracts. The AMM component manages liquidity-based token exchanges, while the order-book component manages user-defined trade orders. Validation conditions in the contracts are used to reject transactions that do not satisfy the required parameters.

The contracts were compiled and tested using Hardhat, with Remix IDE used during development and contract inspection. MetaMask was integrated to allow users to approve and sign transactions from their own wallets. The resulting blockchain state can subsequently be checked independently of the application database.

Frontend and Backend Development

The user interface was developed using React.js to provide a simple and interactive experience. Users

can connect their MetaMask wallet, browse available assets, purchase tokens, and view their transaction history and portfolio. The backend was implemented using Node.js and Express.js, which handle user authentication, process requests, and communicate with both the database and blockchain. MongoDB is used to store user information and asset details that do not need to be recorded on the blockchain, improving the overall efficiency of the application.

Testing and Validation

Testing was performed in multiple stages to verify the interaction between the web application, database, wallet, and blockchain components. The testing process focused on functional correctness rather than production-scale benchmarking.

Functional testing covered user registration, authentication, asset creation, asset listing, wallet connection, token creation, token purchase, token transfer, and ownership updates. Each operation was checked using valid inputs as well as selected invalid conditions to verify that inappropriate requests were rejected.

Smart-contract testing focused on token balances, ownership changes, transaction validation, and the execution of marketplace operations. Particular attention was given to whether the blockchain state after a transaction corresponded to the expected result displayed by the application.

Integration testing examined the communication path between React.js, the Node.js/Express backend, MongoDB, MetaMask, and the Ethereum test environment. This ensured that an operation initiated from the frontend could be processed by the application and, where required, submitted to the blockchain through the user's wallet.

Usability testing was performed by observing the main user workflow, including asset selection, wallet connection, transaction approval, and portfolio verification. The testing indicated that users could complete the primary workflow, although additional guidance may be required for users who are unfamiliar with blockchain wallets.

The evaluation was conducted at the prototype level. No claim is made that the system has been validated for large-scale concurrent usage, production security, regulatory compliance, or commercial deployment.

Decentralized Asset Metadata Binding

The current prototype stores asset descriptions and images in MongoDB as off-chain application data. For a production-oriented implementation, decentralized storage such as IPFS or Arweave can be integrated to provide a content-addressed reference to

the asset metadata and supporting documents. In an IPFS-based workflow, the asset metadata and supporting documents are serialized and uploaded to the network, producing a Content Identifier (CID) derived from the content. The resulting CID can then be stored in the corresponding asset-token smart contract together with the asset identifier. Since the CID is content-addressed, a modification to the referenced content results in a different identifier, allowing the application to detect metadata changes.

An alternative implementation can use Arweave for long-term decentralized data storage. In this approach, the asset metadata and documents are uploaded to Arweave and the resulting transaction reference is associated with the token contract. The application can retrieve the metadata through the stored reference while the blockchain maintains the binding between the tokenized asset and its external metadata.

This decentralized-storage integration is identified as an extension of the current prototype rather than a feature claimed as fully implemented in the evaluated system.

Per-Asset Contract Deployment

A scalable deployment model can associate each tokenized asset with a distinct token-contract address. The deployment process consists of registering the asset, configuring its token name, symbol, supply and issuer, deploying the corresponding token contract, and storing the resulting contract address against the asset record. A factory contract can automate this process by creating a new token-contract instance whenever an administrator registers a new asset. The factory can emit an event containing the asset identifier and newly deployed token-contract address, allowing the backend to associate the on-chain contract with the corresponding MongoDB asset record.

For larger deployments, a factory-based architecture can reduce repetitive deployment logic. Minimal proxy clones are one established approach for creating multiple instances of the same contract implementation from an on-chain factory.

In the current prototype, contract deployment and configuration are handled through the development and deployment workflow using Hardhat. A factory-based deployment model is considered an architectural extension for scalable multi-asset issuance.

V. TECHNICAL STACK

Table I. Technical Stack

Layer / Component	Technology	Primary Function
Presentation Layer	React.js / Next.js	Asset browsing, dashboards, portfolio and transaction interface
Application Layer	Node.js / Express.js	Authentication, REST APIs, application logic and blockchain coordination
Persistence Layer	MongoDB	User profiles, asset metadata, images and application-side records
Blockchain Layer	Solidity / Ethereum Sepolia	Token creation, transfers, ownership state and transaction execution
Wallet Layer	MetaMask / Web3.js	Wallet connection and transaction authorization
Development & Testing	Hardhat / Remix IDE	Contract development, testing, deployment and inspection

The platform uses a layered technology stack in which each component performs a distinct role. React.js/Next.js provides the user interface, while Node.js and Express.js handle application-level services. MongoDB stores frequently accessed off-chain data, whereas Solidity smart contracts deployed on Ethereum Sepolia maintain blockchain-based token state. MetaMask provides wallet-based transaction authorization, and Hardhat and Remix IDE support contract development and testing.

VI. RESULTS AND DISCUSSION

The developed prototype was evaluated by testing the major functional and blockchain-related workflows of the platform. The evaluation covered asset listing, wallet connection, token creation, token purchase, token transfer, ownership verification, marketplace operations, and communication between the application components.

Initial contract development and functional verification were performed using the Hardhat development environment. After the local verification stage, the blockchain workflow was tested against the Ethereum Sepolia test network where applicable. Hardhat was used as the development and deployment framework, while MetaMask was used to connect the user's wallet and authorize blockchain

transactions.

The evaluation showed that the major application components could communicate successfully during the tested workflows. Asset information was retrieved from MongoDB, wallet operations were handled through MetaMask, and blockchain-related ownership information was maintained through smart-contract state. Invalid transaction conditions tested during the evaluation were rejected according to the validation rules implemented in the contracts.

The evaluation should be interpreted as a prototype-level assessment. The tests do not establish production-level scalability, formal smart-contract security, legal validity of tokenized ownership, or compliance with financial and property regulations. These aspects require separate technical, legal, and operational evaluation.

Token Transaction and Ownership Testing

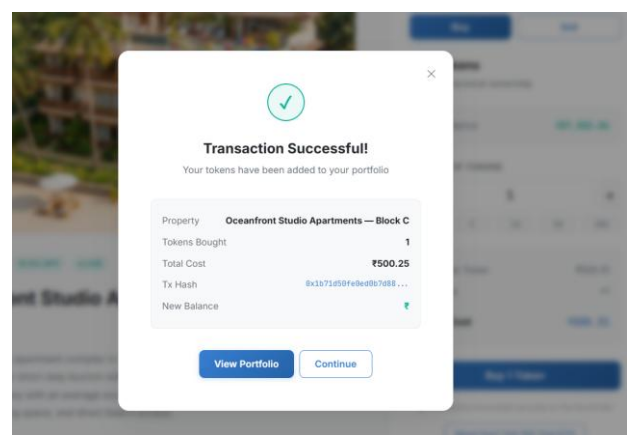


Figure 2. Successful Token Purchase and Transaction Details

Token transaction testing was performed to verify whether token balances and ownership records changed according to the intended transaction. Test cases included successful token transfers as well as transactions containing invalid conditions.

For a successful transaction, the resulting token balance of the sender and recipient was compared with the expected transaction outcome. The corresponding blockchain state was then checked to determine whether the ownership information reflected the completed transfer.

Table II. Functional Test Summary

Test ID	Operation	Expected Outcome	Result
T1	User registration	User record created	Pass

T2	Asset creation	Asset stored and listed	Pass
T3	Token purchase	Token balance updated	Pass
T4	Token transfer	Sender and recipient balances updated	Pass
T5	Invalid transaction	Contract rejects operation	Pass
T6	Ownership verification	Application matches blockchain balance	Pass
T7	Order-book trade	Valid order executes correctly	Pass
T8	AMM swap	Token output follows pool reserves	Pass

Negative test cases were also used to examine the behaviour of the smart contracts when transaction conditions were not satisfied. In these cases, the contract validation logic prevented the state transition from being completed.

The results demonstrate that the implemented token-transfer mechanism maintained consistent blockchain balances for the evaluated test cases. Within the scope of the prototype, the blockchain therefore served as the authoritative source for digital token ownership. This should not be interpreted as establishing legal title to the underlying physical asset, since legal ownership and regulatory recognition require additional off-chain processes.

Transaction Processing Observation

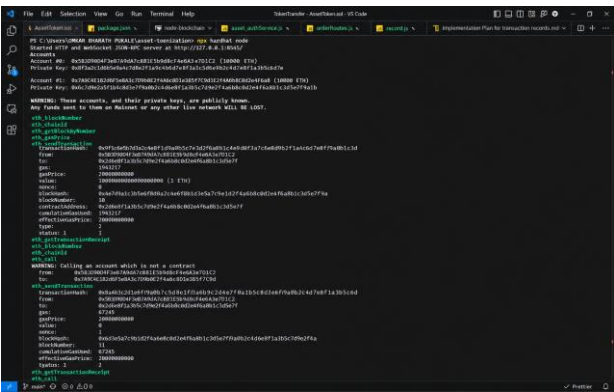


Figure 3. Transaction Processing Observation

Transaction processing was examined across the main application workflow, beginning with user interaction in the frontend and ending with blockchain confirmation. A typical transaction involved selecting an asset, connecting the MetaMask wallet,

submitting the required transaction parameters, approving the transaction through the wallet, and waiting for the blockchain operation to complete.

The evaluation showed that blockchain transactions introduce an additional confirmation step compared with conventional database operations. This behavior is expected because the transaction must be submitted to the blockchain network and included in a block before the resulting state can be treated as confirmed.

The application therefore separates ordinary application data from blockchain-dependent operations. Asset descriptions, images, and user-related application information are maintained in MongoDB, while token balances and blockchain transaction states are associated with smart-contract execution.

Transaction time was observed during testing, but the measured values are dependent on network conditions and should not be treated as a fixed performance characteristic of the system.

Ownership Record Consistency

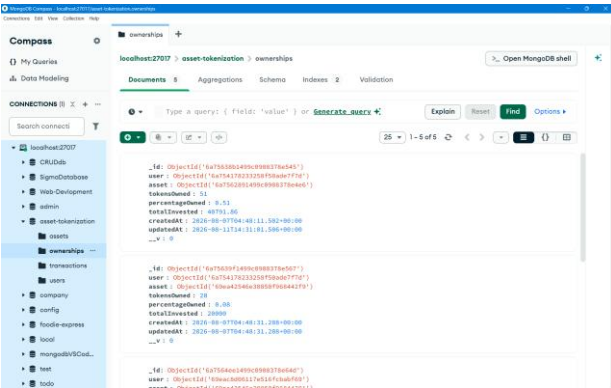


Figure 4. MongoDB Ownership Record Used for Consistency Verification

Ownership consistency was evaluated by comparing the token balances reported by the smart contract with the ownership information presented through the application. Multiple transactions were performed to verify that a transfer from one wallet resulted in the corresponding reduction and increase in token balances.

The test also examined whether repeated or invalid operations could produce inconsistent ownership states. Transactions that failed the contract validation conditions did not result in an unintended change to the token balances observed during the evaluation.

These observations indicate that the blockchain state and the application's ownership display remained consistent for the tested scenarios. The result supports the use of the blockchain as a verifiable record

for digital token balances within the prototype.

Asset Search and Data Retrieval

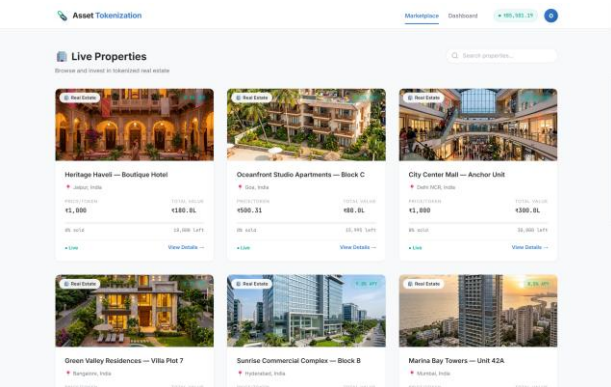


Figure 5. Asset Search and Data Retrieval

The asset retrieval functionality was tested to verify the separation between application-level information and blockchain-based ownership information. Asset descriptions, images, pricing information, and other frequently accessed details were retrieved from MongoDB, while token-related information was obtained through the corresponding blockchain operations.

The test confirmed that the application could retrieve asset information and associate it with the corresponding token-related data displayed to the user. This separation reduces the need to store descriptive application data directly on the blockchain while retaining blockchain-based verification for token ownership and transactions.

The approach also allows the application database to handle frequently accessed content without making the blockchain responsible for storing every application field.

System Integration Testing

Integration testing focused on the interaction between the frontend, backend, database, wallet, and blockchain components. A complete workflow was tested by selecting an asset through the React.js interface, retrieving the required application data from the backend, connecting the MetaMask wallet, submitting a transaction, and checking the resulting blockchain state.

The integration tests were used to identify communication and transaction-handling issues between individual components. Problems encountered during development were corrected and the complete workflow was repeated to verify the resulting behaviour.

The evaluation confirms integration of the major prototype components under the tested conditions. However, the testing did not constitute a large-scale

concurrent-user benchmark; therefore, production-level scalability cannot be inferred from these results.

Limitations

The proposed system is a prototype implementation and therefore has several limitations. First, the token representation used by the platform demonstrates digital fractional ownership within the application but does not by itself establish legal ownership of the corresponding physical asset. Establishing legal rights would require appropriate property documentation, contractual arrangements, identity verification, and compliance with applicable regulations.

Second, the evaluation was performed within a controlled development and test-network environment. The study did not perform large-scale concurrent-user benchmarking or long-duration production testing. Consequently, the observed behaviour cannot be used to claim production-level scalability.

Third, the prototype does not provide a complete KYC/AML, taxation, custody, or regulatory-compliance framework. These requirements are dependent on the asset class, jurisdiction, participants, and operating model of the platform.

Finally, the smart contracts were evaluated through functional testing but were not presented as having undergone a formal independent security audit. Additional security analysis would therefore be required before deployment involving real assets or funds.

VII. CONCLUSION

This work presented a prototype for representing real-world assets as fractional digital tokens using Solidity smart contracts and an integrated web application. The implementation combines React.js, Node.js/Express.js, MongoDB, MetaMask, and Ethereum Sepolia to support asset registration, token issuance, trading, transfer, and ownership verification.

The implementation separates application data from blockchain state. MongoDB is used for asset descriptions, user information, images, and other off-chain data, while token balances and ownership-related transactions are handled through smart contracts. This architecture allows conventional web technologies to be combined with blockchain-based transaction processing within a single application.

The prototype was evaluated through functional, smart-contract, integration, and usability testing. The tested workflows demonstrated that asset information

could be retrieved, wallets could be connected, token transactions could be submitted, and ownership balances could be verified through the blockchain. The evaluation also showed that blockchain transactions introduce confirmation time and additional wallet-interaction steps that must be considered when designing user-facing applications.

The current implementation should be considered a technical prototype rather than a production investment platform. In particular, token ownership within the application should not be interpreted as automatic legal ownership of an underlying physical asset. Regulatory compliance, KYC/AML procedures, asset verification, custody, taxation, and legal documentation would require additional mechanisms outside the present implementation.

Future development can focus on independent smart-contract security auditing, larger-scale performance evaluation, improved wallet usability, support for additional asset categories, stronger identity and compliance mechanisms, deployment on a production blockchain, and more sophisticated secondary-market functionality. The prototype provides a foundation for studying how blockchain-based fractional tokenization can be integrated with conventional web application infrastructure.

VIII. REFERENCES

- [1] Nishith Desai Associates, “Real Estate Tokenisation — Legal, Tax and Regulatory Considerations,” Mar. 2025.
- [2] I. Agur, G. Villegas-Bauer, T. Mancini-Griffoli, M. S. Martinez Peria, and B. Tan, “Tokenization and Financial Market Inefficiencies,” IMF Fintech Note 2025/001, International Monetary Fund, Jan. 2025, doi: 10.5089/9798400298905.063.
- [3] World Economic Forum, “Asset Tokenization in Financial Markets: The Next Generation of Value Exchange,” Insight Report, May 21, 2025.
- [4] U. Soni, O. Fines, and J. Sun, “An Investment Perspective on Tokenization — Part I: A Primer on the Use of Distributed Ledger Technology to Tokenize Real-World and Financial Assets,” CFA Institute, Jan. 22, 2025.
- [5] International Financial Services Centres Authority, “Regulatory Approach towards Tokenization of Real-World Assets,” Consultation Paper, Feb. 26, 2025.
- [6] H. E. Benedetti and G. Rodriguez-Garnica, “Tokenized Assets and Securities,” SSRN, Dec. 2021, doi: 10.2139/ssrn.4069119.